Gmapping Mapping Algorithm

Gmapping Mapping Algorithm

1. Course Content
2. Gmapping Algorithm Introduction
3. Case Demonstration
    3.1 Mapping Process
    3.2 Saving the Map
    3.3 Configuration File
4. Source Code Analysis

1. Course Content

Basic: Run the example program, perform SLAM mapping with the gmapping mapping algorithm, and save the grid map.

Advanced: Understand the gmapping algorithm's configuration parameters.

2. Gmapping Algorithm Introduction

- Gmapping is a commonly used open-source SLAM algorithm based on the filtering SLAM framework.
- Gmapping is based on the RBPF particle filter algorithm, separating the simultaneous localization and mapping process: it performs localization first, then mapping.
- Gmapping makes two main improvements on the RBPF algorithm: an improved proposal distribution and selective resampling.

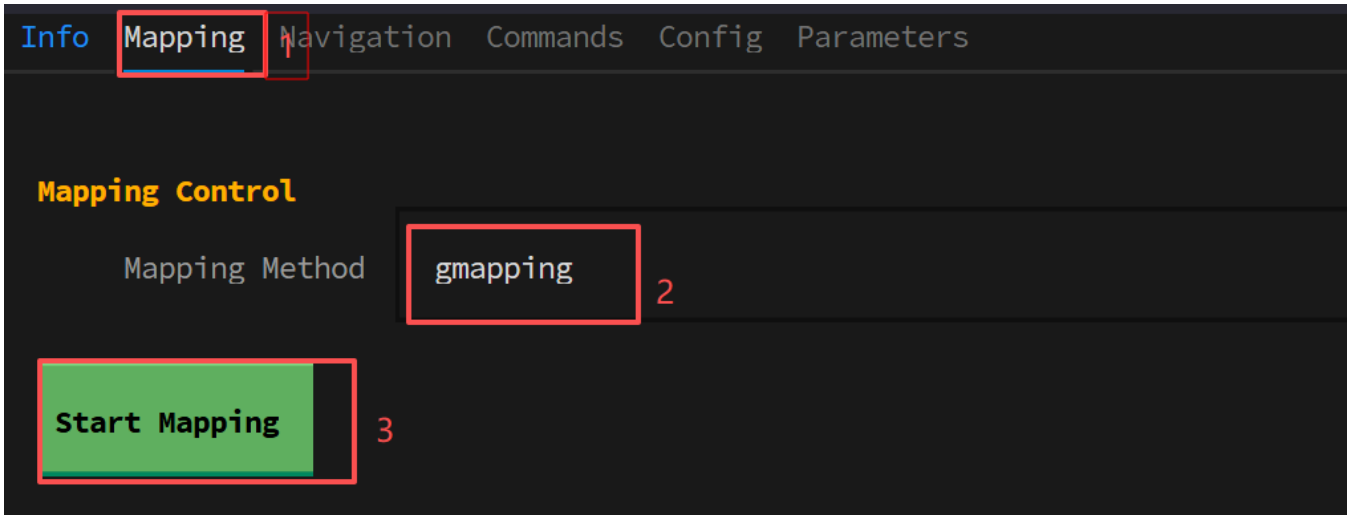
Advantages: Gmapping can build indoor maps in real time, requires relatively little computation for building small-scene maps, and achieves high accuracy.

Disadvantages: As the scene grows larger, the number of particles required increases; because each particle carries a map, the memory and

computation required to build a large map both increase. Therefore, it is not suitable for building large-scene maps. It also lacks loop closure detection, so the map may become misaligned during loop closure. Although increasing the number of particles can make the map close, this comes at the cost of increased computation and memory.

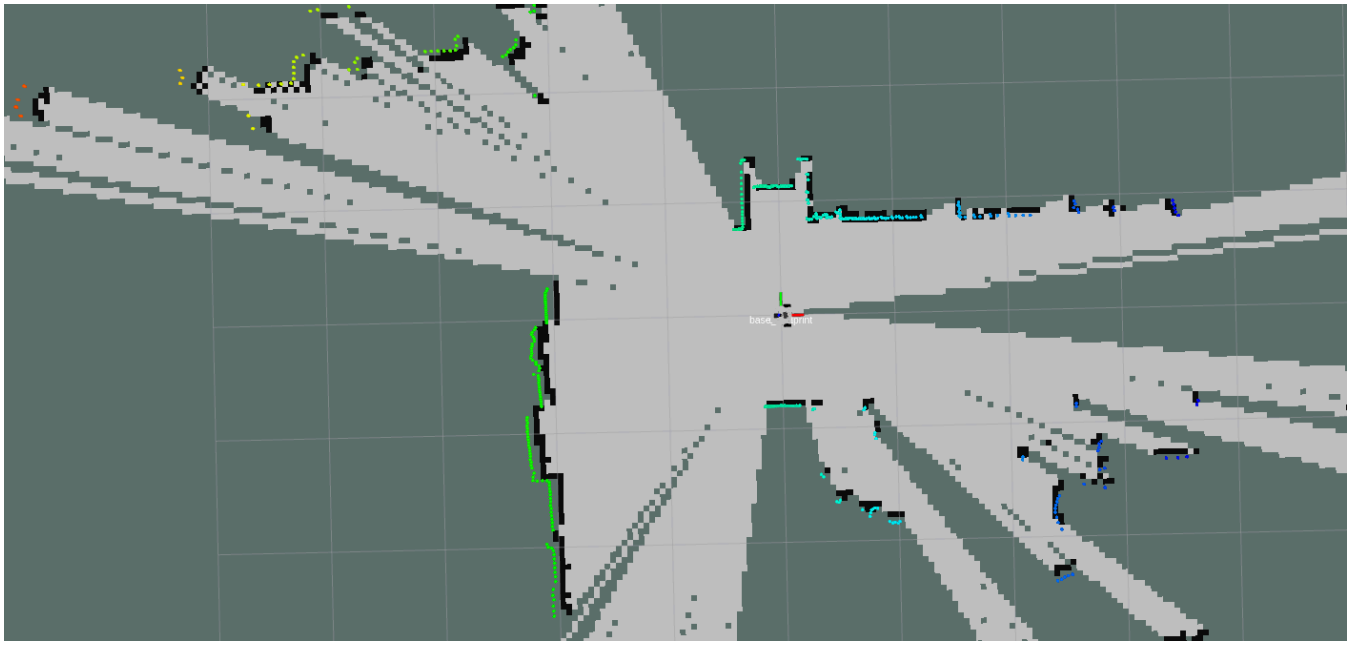
3. Case Demonstration

- Launch method 1: GUI launch



- Launch method 2: CLI command launch in VNC

```
ros2 launch yahboomcar_bringup gmapping.launch.py
```



3.1 Mapping Process

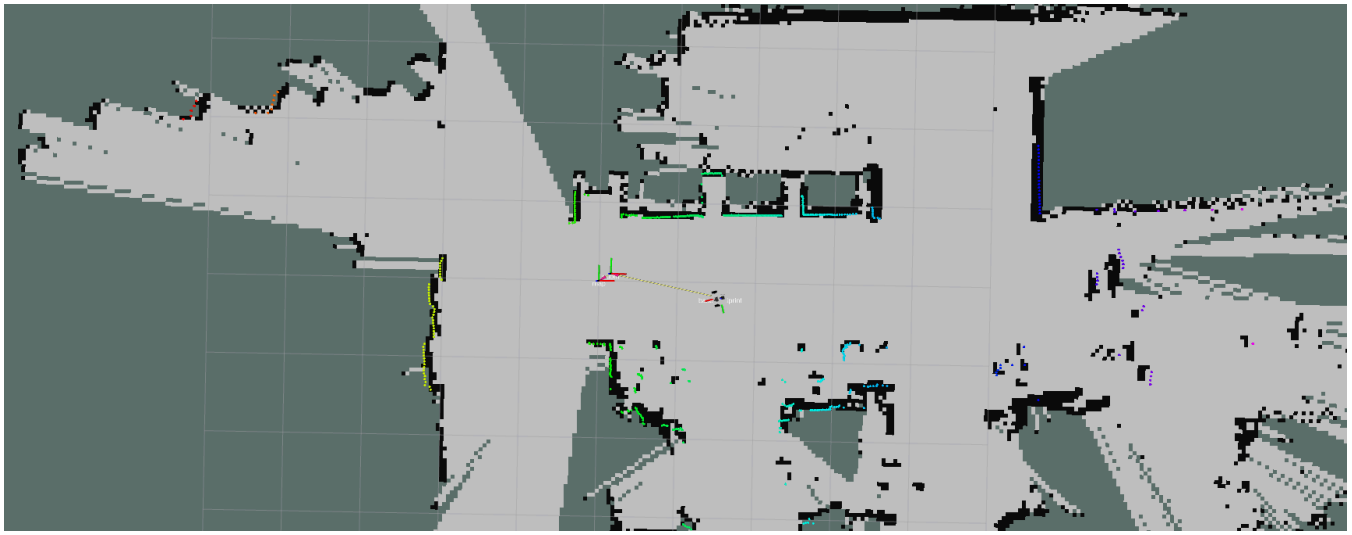
- Note: when mapping, the slower the speed, the better the result (mainly the rotation speed should be slower).
- Control the car's movement to complete mapping of the environment. You can use keyboard control or handle control; here, keyboard control is used as an example

```
ros2 run common_demo keyboard_control
```

```
yahboom@yahboom:~$ ros2 run common_demo keyboard_con
trol

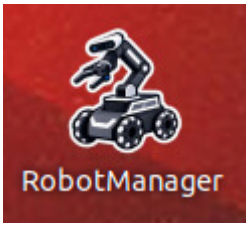
Reading from the keyboard  and Publishing to Twist!
---------------------------
Moving around:
   u    i    o
   j    k    l
   m    ,    .

For Holonomic mode (strafing), hold down the shift k
ey:
```

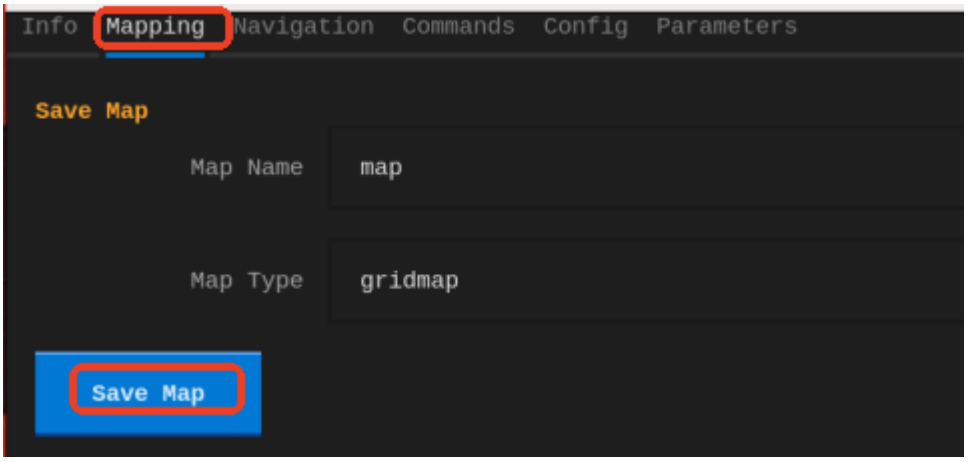


3.2 Saving the Map

- Method 1: operate through the UI; start RobotManager



- Click Save Map. You can also customize the map name and map type. If the terminal indicates that saving failed, simply save again



- Method 2: run the following command in the terminal to save the grid map:

```
ros2 launch yahboomcar_bringup save_map_launch.py
```

> **Tip**
>
> **Optional parameter**:
>
> `map_name`: specifies the name for saving the map; the default value is map
>
> For example, to save a map named test.yaml, you need to add the parameter: `ros2 launch yahboomcar_bringup save_map_launch.py map_name:=test.yaml`

When the terminal displays **Map saved successfully**, the map has been saved successfully. If saving fails, try saving again.

The default map save path is as follows:

```
~/map/map.yaml
```

A pgm image and a yaml file yahboom_map.yaml

```
image: yahboom_map.pgm
mode: trinary
resolution: 0.05
origin: [-10, -10, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

Parameter analysis:

- image: the path of the map file; it can be an absolute path or a relative path
- mode: this attribute can be one of trinary, scale, or raw, depending on the mode selected; trinary mode is the default mode
- resolution: the map's resolution, in meters/pixel

- origin: the 2D pose (x, y, yaw) of the map's lower-left corner; here yaw is rotated counter-clockwise (yaw=0 means no rotation). At present, many parts of the system ignore the yaw value.
- negate: whether to invert the meaning of white/black and free/occupied (the interpretation of thresholds is unaffected)
- occupied_thresh: pixels with an occupancy probability greater than this threshold are considered fully occupied.
- free_thresh: pixels with an occupancy probability less than this threshold are considered fully free.

3.3 Configuration File

- Configuration file path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/slam_gmapping.yaml
```

- Meaning of the default configuration parameters:

```
slam_gmapping:
  ros__parameters:
    # Angular update threshold (radians): update the map when the robot rotates beyond this
angle
    angularUpdate: 0.25
    # Angular sampling step (radians): the angular search step used for scan matching
    astep: 0.05
    # Name of the robot base coordinate frame
    base_frame: base_footprint
    # Name of the map coordinate frame
    map_frame: map
    # Name of the odometry coordinate frame
    odom_frame: odom
    # Iteration termination condition of the scan matching algorithm: stop iterating when
the parameter change is less than this value
    delta: 0.05
    # Maximum number of iterations for scan matching
    iterations: 5
    # Kernel size used for scan matching; affects the smoothness of the match
    kernelSize: 1

    # Distance sampling range for large-scale scan matching (meters)
    lasamplerange: 0.005
    # Distance sampling step for large-scale scan matching (meters)
    lasamplestep: 0.005
    # Linear update threshold (meters): update the map when the robot moves beyond this
distance
    linearUpdate: 0.5
    # Distance sampling range for small-scale scan matching (meters)
    llsamplerange: 0.01
    # Distance sampling step for small-scale scan matching (meters)
    llsamplestep: 0.01
```

```yaml
    # Measurement noise covariance of the laser scan
    lsigma: 0.075
    # Number of laser beams to skip: 0 means using all laser points
    lskip: 0
    # Linear sampling step (meters): the linear search step used for scan matching
    lstep: 0.05
    # Map update interval (seconds)
    map_update_interval: 3.0
    # Maximum laser range (meters): points beyond this distance are ignored
    maxRange: 6.0
    # Effective maximum laser range (meters): the maximum distance used for mapping
    maxUrange: 4.0
    # Minimum score threshold for scan matching: matches below this value are discarded
    minimum_score: 0.0
    # Occupancy probability threshold: cells greater than this value are considered occupied
    occ_thresh: 0.25
    # Gain factor: affects the speed of map updates
    ogain: 3.0
    # Number of particles in the particle filter
    particles: 30
    # QoS (Quality of Service) parameter settings
    qos_overrides:
      /parameter_events:
        publisher:
          depth: 1000          # Message queue depth
          durability: volatile # Durability policy
          history: keep_all    # History policy
          reliability: reliable # Reliability policy
      /tf:
        publisher:
          depth: 1000
          durability: volatile
          history: keep_last   # Keep only the latest message
          reliability: reliable
    # Resampling threshold: trigger resampling when the number of effective particles falls
below this ratio
    resampleThreshold: 0.5
    # Angular noise covariance of scan matching
    sigma: 0.05
    # Odometry model parameter: proportion of linear error caused by linear motion
    srr: 0.1
    # Odometry model parameter: proportion of angular error caused by linear motion
    srt: 0.2
    # Odometry model parameter: proportion of linear error caused by angular motion
    str: 0.1
    # Odometry model parameter: proportion of angular error caused by angular motion
    stt: 0.2
    # Time update interval (seconds): update the map even if the robot has not moved
    temporalUpdate: 1.0
    # Coordinate transform publishing period (seconds)
    transform_publish_period: 0.05
    # Whether to use simulation time
    use_sim_time: false
```

```
    # Maximum x-axis range of the map (meters)
    xmax: 100.0
    # Minimum x-axis range of the map (meters)
    xmin: -100.0
    # Maximum y-axis range of the map (meters)
    ymax: 100.0
    # Minimum y-axis range of the map (meters)
    ymin: -100.0
```

4. Source Code Analysis

- gmapping mapping launch file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/gmapping.launch.py
```

- `slam_gmapping_config_file`: the full path for loading the gmapping parameter configuration file `slam_gmapping.yaml`
- `rviz_cmd`: declares the `rviz` launch parameter, default value `True`, used to control whether to automatically start Rviz2
- `bringup_car_base`: includes `car_base.launch.py`, enabling the LiDAR (`use_lidar: True`) and IMU (`use_imu: True`), disabling the camera and magnetometer, and using the EKF odometry as the TF source (`odom_tf_source: ekf`)
- `gmapping_node`: starts the `slam_gmapping` node and loads the above configuration file to perform mapping
- `rviz_node`: starts Rviz2 and loads the preset configuration `gmapping_rviz.rviz`, controlled by the `rviz` parameter
- `LaunchDescription`: aggregates and returns all declared parameters and nodes

```
def generate_launch_description():

    slam_gmapping_config_file =
os.path.join(get_package_share_directory("yahboomcar_bringup"), "param",
"slam_gmapping.yaml")

    rviz_cmd=DeclareLaunchArgument(
        "rviz",
        default_value="True",
        description="Start Rviz2 automatically ?",
    )

    bringup_car_base=IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(get_package_share_directory('yahboomcar_bringup'
),'launch','car_base.launch.py')),
            launch_arguments={'log_level': 'info',
```

```
                                'use_imu': 'True',
                                'use_lidar': 'True',
                                'use_mag': 'False',
                                'use_camera': 'False',
                                'odom_tf_source': 'ekf'
                                }.items())

    gmapping_node=Node(
    package='slam_gmapping',
    executable='slam_gmapping',
    output='screen',
    parameters=[slam_gmapping_config_file]
    )
    rviz_node=Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        arguments=['-d', os.path.join(get_package_share_directory('yahboomcar_bringup'),
'rviz', 'gmapping_rviz.rviz')],
        condition=IfCondition(LaunchConfiguration('rviz'))
    )

    return LaunchDescription([
            rviz_cmd,
            rviz_node,
            bringup_car_base,
            gmapping_node,
    ])
```

- Save map command launch file path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/save_map_launch.py
```

> **Important**
>
> In subsequent chapter courses, the save map command and the command source code in RobotManager's save map feature are all save_map_launch.py, which switches between different functions through launch parameters. save_map_launch.py will not be explained again in subsequent courses

- `map_folder_dir`: defines the map save directory `~/map`
- `declared_arguments`: declares two launch parameters, `map_name` (map name, default `map`, without file extension) and `map_type` (map type, default `gridmap`, options `gridmap / pbstream / posegraph`)
- `map_path / pbstream_path`: concatenates the save directory and map name into the full path
- `save_map`: uses `nav2_map_server`'s `map_saver_cli` to save the grid map, specifying a free space threshold of `0.196` and an occupied threshold of `0.65`; executes only when `map_type == 'gridmap'`
- `save_pbstream`: calls Cartographer's `/write_state` service via `ros2 service call` to save a map in

`.pbstream` format; executes only when `map_type == 'pbstream'`

- `save_posegraph`: calls slam_toolbox's `/slam_toolbox/serialize_map` service via `ros2 service call` to save the pose graph; executes only when `map_type == 'posegraph'`
- `LaunchDescription`: aggregates and returns all parameter declarations and the three save nodes (executed conditionally)

```
def generate_launch_description():
    map_folder_dir = os.path.join(os.path.expanduser('~'), 'map')

    declared_arguments = [
        DeclareLaunchArgument(
            'map_name',
            default_value='map',
            description='Map name to save, without file extension',
        ),
        DeclareLaunchArgument(
            'map_type',
            default_value='gridmap',
            description='Type of map to be saved (optional):gridmap、pbstream、posegraph',
        ),
    ]

    map_name = LaunchConfiguration('map_name')
    map_type = LaunchConfiguration('map_type')
    map_path = PathJoinSubstitution([map_folder_dir, map_name])
    pbstream_path = PathJoinSubstitution([map_folder_dir, map_name])

    save_map = Node(
        package='nav2_map_server',
        executable='map_saver_cli',
        arguments=[
            '-f', PathJoinSubstitution([map_folder_dir, map_name]),
            '--free', '0.196',
            '--occ', '0.65'],
        condition=IfCondition(PythonExpression(["'", map_type, "' == 'gridmap'"]))
    )


    save_pbstream = ExecuteProcess(
        condition=IfCondition(PythonExpression(["'", map_type, "' == 'pbstream'"])),
        cmd=[
            'ros2', 'service', 'call',
            '/write_state',
            'cartographer_ros_msgs/srv/WriteState',
            ['{filename: "', pbstream_path, '.pbstream"}'],
        ],
        shell=False,
        emulate_tty=True,
        output='screen',
    )

    save_posegraph = ExecuteProcess(
```

```
        condition=IfCondition(PythonExpression(["'", map_type, "' == 'posegraph'"])),
        cmd=[
            'ros2', 'service', 'call',
            '/slam_toolbox/serialize_map',
            'slam_toolbox/srv/SerializePoseGraph',
            ['{filename: "', map_path, '"}'],
        ],
        shell=False,
        emulate_tty=True,
        output='screen',
    )


    return LaunchDescription([
        *declared_arguments,
        save_map,
        save_pbstream,
        save_posegraph,
    ])
```